

Khulna University of Engineering & Technology

Department of Computer Science and Engineering

PROJECT REPORT

**Title: Interpretable Sentiment Classification in
Software Engineering Communication**

Course No: CSE 4122

Course Title: Natural Language Processing Laboratory

Submission Date: 23 September, 2026

Submitted To:

Dr. K. M. Azharul Hasan

Professor, Dept. of CSE, KUET

Md Nazirulhasan Shawon

Assistant Professor, Dept. of CSE, KUET

Submitted By:

Al Shariar Hossain (2107066)

Hassan Mohammed Naquibul Hoque (2107077)

Contents

1	Introduction	3
1.1	Contributions	3
2	Related Work	4
3	System Architecture	4
3.1	Pipeline Overview	4
3.2	Module Architecture	5
4	Data Acquisition	5
4.1	Corpus: Senti4SD Gold Standard	5
4.2	Corpus Statistics	5
5	Text Extraction and Cleaning	6
5.1	Critical Order Decisions	6
6	Preprocessing	6
6.1	Normalisation Modes	6
6.2	Normalisation Ablation	7
6.3	Data Splitting	7
7	Feature Engineering	7
7.1	TF-IDF Representations	7
7.2	The Compound-Phrase Finding	8
7.3	Embeddings	8
8	Model Building	9
8.1	The Model Grid	9
8.2	Hyperparameter Tuning	9
8.3	Why a Linear Model and Not a Transformer	9
9	Evaluation and Results	11
9.1	Baselines	11
9.2	Test-Set Results	11
9.3	Key Findings	11
9.4	Error Analysis	12
10	Explainability Analysis	12
10.1	LIME Explanations	12
10.2	SHAP Explanations	12
10.3	LIME–SHAP Agreement	13
10.4	Stability Analysis	13
11	Stress Test — The Central Result	13
11.1	The Headline Result	13
11.2	Outcome #3 — Lexical Bias Tracks Pretraining	14
12	Web Application	14
12.1	Held-Out Probe Sentences	14
13	Limitations	15

14 Conclusion**16**

1 Introduction

Sentiment analysis has become an increasingly important tool in software engineering (SE) for tracking developer morale, triaging hostile code reviews, monitoring community health on platforms like StackOverflow, and understanding the emotional tone of technical discussions. However, deploying general-purpose sentiment analysis models in the SE domain introduces a fundamental challenge: **domain shift**.

General-purpose sentiment classifiers are typically trained on product reviews, movie reviews, or social-media posts, where words such as *crashed*, *killed*, *failed*, and *fatal* reliably indicate displeasure or negativity. In software engineering communication, these very same words are part of routine technical vocabulary — “*Kill the process before restarting the server*” or “*The build failed with a fatal error on line 42*” are entirely neutral, factual descriptions of everyday operations. A model that has not learned this distinction will systematically mistake technical description for hostility, producing false alarms that undermine the utility of sentiment-aware development tools.

This project addresses the domain-shift problem through two complementary lenses. First, we build and compare **fifteen sentiment classifiers** spanning five text representations (three TF-IDF n -gram configurations, GloVe pretrained embeddings, and corpus-trained Word2Vec embeddings) crossed with three classifier families (Naïve Bayes as the generative baseline, Logistic Regression, and Linear SVM as discriminative alternatives). Second, and more importantly, we go beyond accuracy measurement and apply **model explainability** techniques — LIME (Local Interpretable Model-agnostic Explanations) and SHAP (SHapley Additive exPlanations) — to audit *why* models succeed or fail on technical text. This transforms the evaluation from “the model is 83% accurate” into “the model treats *afraid* and *hate* as negative evidence and does not key on technical vocabulary.”

Our pipeline is built on the **Senti4SD** gold-standard corpus [1], comprising 4,331 human-annotated StackOverflow posts labelled as negative, neutral, or positive with substantial inter-annotator agreement (Fleiss’ $\kappa = 0.7586$). The entire system is implemented as a reproducible six-stage pipeline where each stage produces artefacts consumed by the next, every number in this report traces back to a runnable command, and the test set is read exactly once after all design decisions are final.

A critical finding that emerges early and is confirmed by four independent analyses is that the Senti4SD corpus **does not contain the harsh technical vocabulary** the research question is about — *fatal*, *hang*, and *abort* occur zero times in the training split, and *kill* only three times. To address this, we construct a purpose-built **60-sentence stress test** containing harsh-but-neutral SE sentences, genuine complaints, and minimal pairs that isolate technical vocabulary from real hostility. On this stress set, the bias becomes unambiguous: VADER (a general-purpose lexicon) labels 70% of neutral SE sentences as negative, while the false-alarm rate falls monotonically with how much general English the representation carries — GloVe 40%, Word2Vec 28%, TF-IDF 14%.

1.1 Contributions

1. A fully reproducible six-stage pipeline over the Senti4SD corpus, producing 15 trained models, with one command per stage.
2. Empirical measurement — not assumption — of whether stemming and lemmatisation help or harm sentiment classification in the SE domain.
3. Evidence, from four independent analyses, that the standard SE sentiment corpus largely lacks the technical vocabulary that motivates SE sentiment research.
4. A 60-sentence stress test isolating technical vocabulary from hostility, yielding a clean ordering of lexical bias across representations.
5. Application of LIME and SHAP to audit model behaviour at the token level, with measured inter-method agreement (Spearman $\rho = 0.872$).

2 Related Work

SE-specific sentiment analysis. Calefato et al. [1] established the Senti4SD benchmark, demonstrating that general-purpose sentiment tools transfer poorly to developer communication and releasing the gold standard used in this project. Subsequent work, including SentiCR [6] for code review and SentiSE, has reinforced this finding across different SE communication channels. Our contribution differs: rather than building a better classifier, we provide an *audit* of what words the model actually uses.

Explainability methods. LIME [2] generates local explanations by fitting a linear surrogate model to perturbations of a single input. SHAP [3] provides theoretically grounded feature attributions based on Shapley values from cooperative game theory, with exact computation available for linear models. Both have become standard for text classification explanations. In this work, we treat their known instability as a measured quantity (mean Jaccard overlap 0.907 across seeds) rather than merely a caveat.

Word embeddings and domain shift. Pennington et al. [4] introduced GloVe embeddings trained on large general-English corpora. Mikolov et al. [5] proposed Word2Vec for learning word representations from local context. Our nearest-neighbour analysis demonstrates directly that GloVe places *kill* beside *shoot* and *poison*, and *abort* beside *fetuses* — a general-English model reading StackOverflow genuinely sees violence and medical contexts rather than process management.

3 System Architecture

3.1 Pipeline Overview

The project is organised as a six-stage sequential pipeline. Each stage produces files consumed by the next, ensuring full reproducibility and traceability.

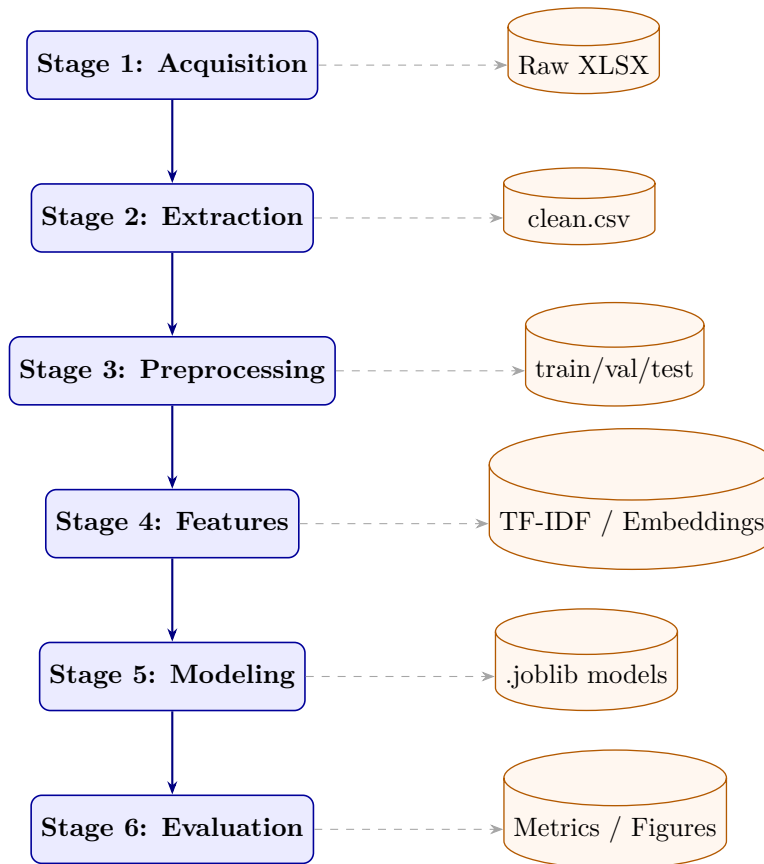


Figure 1: Six-stage pipeline: each stage produces artefacts consumed by the next.

3.2 Module Architecture

Figure 2 shows the modular organisation of the `src/` package, mapping each module to its pipeline stage.

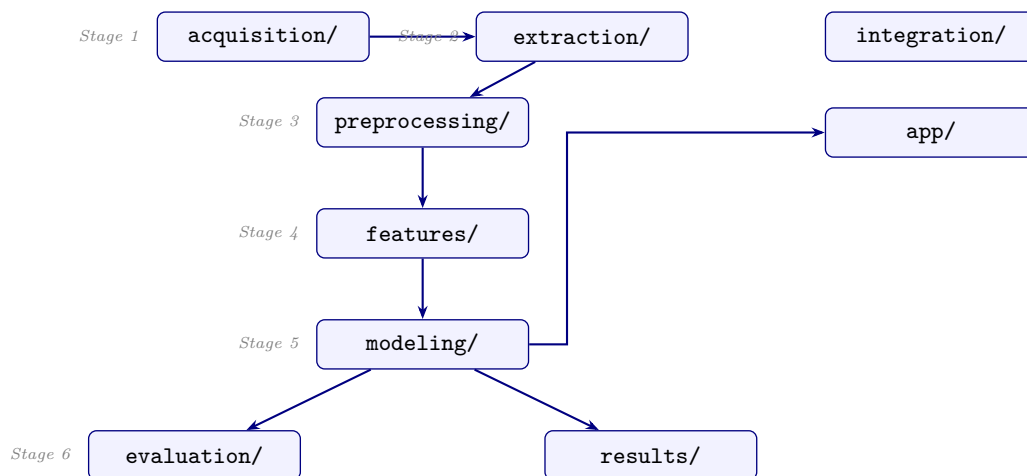


Figure 2: Module architecture of the `src/` package, with each module mapped to its pipeline stage. The `app/` module provides the Flask web interface.

4 Data Acquisition

4.1 Corpus: Senti4SD Gold Standard

The primary dataset is the Senti4SD gold standard [1], consisting of human-annotated StackOverflow posts labelled for sentiment polarity. The gold-standard workbook (`Senti4SD_GoldStandard_EmotionPolarity.xlsx`) is used rather than Senti4SD’s pre-partitioned CSV files because it contains the individual rater columns (`r1`, `r2`, `r3`) required for computing inter-annotator agreement.

4.2 Corpus Statistics

Table 1: Corpus construction summary.

Statistic	Value
Rows loaded from workbook	4,423
Duplicate texts dropped	92
Final corpus size	4,331
Fleiss’ κ	0.7586 (4,359 items, 3 raters)
Mean document length	29.97 words
Mean sentences per document	2.33

Table 2: Class distribution in the Senti4SD corpus.

Class	Count	Share
Negative	1,176	27.15%
Neutral	1,662	38.37%
Positive	1,493	34.47%

The rater columns contain free-text labels with mixed case and typos (`Postive`, `Poitive`, `Netural`); these are repaired programmatically before computing κ , and items with unparseable rater values are excluded.

Implementation: `src/acquisition/download.py` fetches the data; `src/extraction/build_se.py` performs extraction and quality reporting.

5 Text Extraction and Cleaning

A single function, `clean(text, domain)`, processes all text identically across models. The cleaning pipeline applies the following transformations in strict order:

1. **HTML unescaping** — handles double-escaped entities (`<`).
2. **Lowercasing** — applied *second*, so uppercase placeholders inserted in subsequent steps remain distinguishable from ordinary words.
3. **URL replacement** — `https?://\S+` → URL.
4. **Mention replacement** — `@user` → USER.
5. **SE domain rules** — code tags, backtick spans, function calls `foo(bar)`, and dotted paths `a.b.c` → CODE.
6. **Emoticon replacement** — 17 positive patterns → EMO_POS; 14 negative patterns → EMO_NEG.
7. **Negation expansion** — `can't` → `can not`, `won't` → `will not`, etc.
8. **Repeated character collapse** — `soooo` → `soo`.
9. **Whitespace normalisation** — multiple spaces → single space.

5.1 Critical Order Decisions

Two ordering choices are load-bearing and merit explicit documentation:

- **Lowercasing before domain rules:** Placeholders (URL, CODE, EMO_POS) remain uppercase. In the training split, the placeholder CODE occurs in 174 posts and the English word *code* in 303; merging them would produce one meaningless feature spanning 477 posts.
- **URLs before emoticons:** The string `http://` contains `:/`, which matches a sad-face emoticon pattern. The reverse order would transform every URL into `httpEMO_NEG/...`.

Stopword removal is disabled permanently. Negation words (*not*, *no*, *never*) appear in every standard stopwords list, and negation flips sentiment.

Implementation: `src/extraction/clean_text.py` (30 unit tests). The function `describe_rules()` generates the rule table programmatically so the report documentation cannot drift from the code.

6 Preprocessing

6.1 Normalisation Modes

Three normalisation modes are implemented behind a single interface, `normalize(text, mode)`:

- **"none"** — tokenise only; words retain their inflections.

- **"lemma"** — POS-aware lemmatisation via `WordNetLemmatizer` with tags from `nltk.pos_tag`. Without POS tags, `lemmatize("stopped")` returns `stopped` (assuming noun), defeating the purpose.
- **"stem"** — Porter stemming, which is cruder but merges more aggressively (*finally* → *final*).

All three modes use the **same tokeniser** (`TOKEN_PATTERN`), ensuring the only variable is the word-normalisation step. Placeholders are never touched — stemming `CODE` would fold it into the English noun *code*.

6.2 Normalisation Ablation

Rather than assuming whether normalisation helps, we **measured** it. The same model (TF-IDF (1,2) + Logistic Regression at defaults) was trained three times, scored on **validation**:

Table 3: Normalisation ablation results on the validation split (650 documents).

Mode	Vocabulary	Accuracy	Macro-F1	neut→neg
none	12,723	0.7938	0.7863	10.4%
lemma	12,186	0.7831	0.7754	11.2%
stem	12,371	0.7908	0.7844	10.0%

Selected: none. The gap between **none** and **stem** is 0.0019 macro-F1, roughly 2 of 650 documents, well inside the noise floor. The defensible justification is that **none** preserves the inflections the project studies and is simplest. Notably, the two metrics **disagree**: **none** wins on macro-F1 while **stem** has the lowest neutral→negative rate — a miniature of the project’s central tension between accuracy and interpretability.

6.3 Data Splitting

Stratified 70/15/15 splits are created with `random_state=42`:

Table 4: Data split sizes.

Split	Documents
Train	3,031
Validation	650
Test	650

Splits are frozen **before** any model is built. Every vectoriser, embedding, and scaler is fit on **train only**; fitting on anything else would leak the test set.

Implementation: `src/preprocessing/normalize.py` (29 tests), `src/preprocessing/splits.py`.

7 Feature Engineering

7.1 TF-IDF Representations

Three TF-IDF configurations are constructed, all sharing `min_df=2`, `max_df=0.95`, `sublinear_tf=True`, and the project’s unified `TOKEN_PATTERN`. Each vectoriser embeds the `clean()` and `normalize()` functions as its preprocessor, so a saved model is a complete pipeline from raw text to prediction.

Table 5: TF-IDF vocabulary statistics (fitted on training split only).

Config	<i>n</i> -grams	Vocabulary	Sparsity	Features/doc
tfidf11	(1,1)	3,449	0.9931	23.7
tfidf12	(1,2)	12,723	0.9968	40.7
tfidf13	(1,3)	17,902	0.9974	46.7

Most distinctive terms per class (scored by mean weight inside the class minus mean weight outside):

Table 6: Top distinctive terms per sentiment class.

Class	Top terms
Negative	EMO_NEG, sad, hate, horrible, afraid, terrible
Neutral	?, URL, how, what, use, see
Positive	!, excellent, thanks, great, EMO_POS, <i>thanks!</i>

7.2 The Compound-Phrase Finding

The proposal motivates *n*-grams with SE-specific phrases: *fatal error*, *kill the process*, *null pointer exception*. A critical finding is that **none of these occurs even once** in the training corpus. Of 14 tested phrases, the 5 absent ones are exactly the canonical jargon examples. What *n*-grams actually capture is **negation** (*does not*: 124 posts; *not work*: 34) and **politeness** (*thank you*: 54 positive, 0 elsewhere).

7.3 Embeddings

Table 7: Embedding statistics.

Embedding	Vocab.	OOV tokens	Empty docs	Placeholders
GloVe (pretrained, 100d)	400,000	2.4%	9	none known
Word2Vec (self-trained, 100d)	3,752	3.9%	0	all 5 known

Nearest-neighbour analysis — the clearest evidence of domain shift:

Table 8: Nearest neighbours reveal domain shift in GloVe embeddings.

Word	Freq.	GloVe neighbours	W2V neighbours
kill	3	killing, shoot , poison	activities, clarify
crash	9	plane , collision , airplane	bus, modifying
error	65	errors, mistake, fault	assertion , failed , throws
fatal	0	deadly, poisoning	OOV
abort	0	aborted , fetuses	OOV

GloVe places *kill* beside *shoot* and *poison*, *abort* beside *fetuses*, *crash* beside *airplane*. **A general-English model reading StackOverflow genuinely sees violence and aviation disasters.** Word2Vec’s neighbours are mostly noise due to low word frequencies, but *error* (65×) yields sensible domain neighbours, proving the mechanism works given sufficient data.

Implementation: `src/features/features_tfidf.py`, `src/features/features_embed.py` (20 tests combined).

8 Model Building

8.1 The Model Grid

Fifteen models are constructed by crossing five representations with three classifier families:

Table 9: The 15-model grid.

Representation	Description	Classifiers
tfidf11 (1,1)	Unigrams only	MNB, LR, SVM
tfidf12 (1,2)	Unigrams + bigrams	MNB, LR, SVM
tfidf13 (1,3)	Uni- + bi- + trigrams	MNB, LR, SVM
GloVe (100d)	Pretrained embeddings	GNB, LR, SVM
Word2Vec (100d)	Corpus-trained embed.	GNB, LR, SVM

Plus two baselines: majority vote and VADER. Two implementation constraints:

1. **LinearSVC has no `predict_proba`**, so it is wrapped in `CalibratedClassifierCV(cv=5)` — LIME, SHAP, and the web demo all require probability outputs.
2. **MultinomialNB cannot accept embeddings** (which contain negative values), so `GaussianNB` is used for embedding-based generative models.

8.2 Hyperparameter Tuning

All tuning is by `GridSearchCV` with stratified 5-fold cross-validation, scoring by `f1_macro`:

- **MNB:** $\alpha \in \{0.01, 0.1, 0.5, 1.0\}$
- **GNB:** `var_smoothing` $\in \{10^{-9}, 10^{-7}, 10^{-5}, 10^{-3}\}$
- **LR:** $C \in \{0.01, 0.1, 1, 10, 100\}$, `class_weight` $\in \{\text{None}, \text{balanced}\}$
- **SVM:** $C \in \{0.01, 0.1, 1, 10\}$, `class_weight` $\in \{\text{None}, \text{balanced}\}$

Every saved model is a **complete sklearn pipeline** taking raw strings and exposing both `predict()` and `predict_proba()`, stored as `models/se/{features}_{classifier}.joblib`.

Implementation: `src/modeling/models_tfidf.py`, `src/modeling/models_embed.py`.

8.3 Why a Linear Model and Not a Transformer

A fine-tuned transformer such as BERT would almost certainly score higher than 0.8275 macro-F1 on this corpus, and its absence from the model grid is a deliberate design decision rather than an oversight. The reason is that this project’s deliverable is not a leaderboard number but an *audit*: the research question asks *which words* a model relies on, and that question is answerable exactly on a linear model and only approximately on a transformer. Table 10 sets out the trade-off.

Table 10: Why explaining BERT is harder than explaining a linear SVM.

Aspect	Linear SVM (what we have)	BERT (contextual transformer)
Explainability faithfulness	Mathematically exact: SHAP is closed-form for a linear model ($\phi_i = w_i(x_i - \mathbb{E}[x_i])$). Zero approximation error.	Approximation: requires gradient paths or perturbation sampling (LIME / Kernel SHAP); the explanation is itself a fitted model.
Tokenisation	Whole words and n -grams; <i>fatal error</i> is a single feature whose weight can be read directly off the coefficient vector.	Subword pieces (e.g. ['kill', '##ing']); uncommon jargon is split into fragments that must be re-aggregated before a word-level importance even exists.
Compute cost for explanations	SHAP values for the entire test set compute in seconds on a CPU.	LIME / SHAP need hundreds of forward passes <i>per sentence</i> , taking hours without a dedicated GPU.
Masking artefacts	Not applicable — removing a feature is exact, not simulated.	Masking words to test their importance creates out-of-distribution sentences that the model was never trained to read, so the measured importance partly reflects the masking, not the word.

Three consequences are specific to this project and decided the matter:

1. **The exactness is load-bearing, not cosmetic.** The headline explainability result of Section 10 is the Spearman agreement $\rho = 0.872$ between LIME and SHAP. That number is interpretable only because SHAP is exact on a linear model: the correlation measures LIME’s sampling noise against a known ground truth. On a transformer both sides would be approximations, and their agreement would no longer distinguish “the methods describe the model” from “the methods share a sampling bias.”
2. **Subword tokenisation attacks the research question directly.** The phenomenon under study is *word-level* lexical bias — whether *kill*, *fatal*, and *abort* are read as hostility. WordPiece splits exactly this kind of low-frequency jargon into fragments, so the quantity being measured would first have to be reconstructed by a heuristic aggregation step that the linear model does not need.
3. **The stress-test ladder is a controlled comparison.** The central result of Section 11 — false alarms falling monotonically from VADER 70% to GloVe 40% to Word2Vec 28% to TF-IDF 14% — works because each representation carries a *known and ordered* amount of general English. BERT is the natural next rung on that same axis — a measurement this ladder sets up rather than one it is missing: adding it would extend the result, not invalidate it.

In summary. *Could* a transformer be used? Yes — it is the ideal candidate for testing whether contextual attention eliminates the vocabulary bias this project measures. *Why was it not used here?* Because a linear model gives a transparent, exact baseline on which vocabulary bias can be isolated mathematically, without the confounds of deep attention and expensive sampling-based approximation. Establishing that baseline is a precondition for interpreting what a transformer would later change.

9 Evaluation and Results

9.1 Baselines

Table 11: Baseline performance on the test set.

Model	Accuracy	Macro-F1	neut→neg
Majority vote	0.385	0.185	—
VADER	0.715	0.708	25.2%

VADER — a general-purpose lexicon that has never seen a StackOverflow post — calls **one in four** genuinely neutral posts negative. This is the problem statement, measured, before any model is built.

9.2 Test-Set Results

The test split is read **once**, after all design decisions are final. Table 12 shows all 15 models plus baselines:

Table 12: Full test-set results, sorted by macro-F1.

Model	Kind	Macro-F1	Accuracy	neut→neg	neg recall
tfidf13_svm	discrim.	0.8275	0.8308	14.4%	76.1%
tfidf13_lr	discrim.	0.8233	0.8262	14.8%	76.7%
tfidf12_lr	discrim.	0.8108	0.8138	14.0%	75.0%
tfidf12_svm	discrim.	0.8103	0.8154	12.8%	69.3%
tfidf11_svm	discrim.	0.8081	0.8123	12.0%	71.0%
tfidf11_lr	discrim.	0.8070	0.8092	13.6%	75.0%
tfidf12_mnb	gen.	0.7601	0.7662	12.0%	62.5%
tfidf13_mnb	gen.	0.7546	0.7600	13.2%	61.9%
tfidf11_mnb	gen.	0.7331	0.7400	8.4%	56.2%
w2v_svm	discrim.	0.7059	0.7138	11.6%	55.1%
glove_lr	discrim.	0.7054	0.7077	17.2%	63.1%
w2v_lr	discrim.	0.7020	0.7046	20.4%	64.8%
glove_svm	discrim.	0.6936	0.7015	12.4%	52.8%
w2v_gnb	gen.	0.5813	0.5769	49.6%	89.2%
glove_gnb	gen.	0.5668	0.5677	39.2%	79.5%
<i>VADER</i>	<i>baseline</i>	<i>0.7079</i>	<i>0.7154</i>	<i>25.2%</i>	—
<i>Majority</i>	<i>baseline</i>	<i>0.1852</i>	<i>0.3846</i>	—	—

9.3 Key Findings

Research Outcome #1 — Which representation handles SE vocabulary best?

Table 13: Best and mean macro-F1 by representation family.

Representation	Best	Mean
TF-IDF	0.8275	0.7928
Word2Vec	0.7059	0.6631
GloVe	0.7054	0.6553

TF-IDF wins decisively. The n -gram range barely matters (`tfidf11_lr` 0.8070 vs `tfidf13_lr` 0.8233), consistent with the finding that compound SE phrases are absent from the corpus.

Research Outcome #2 — Generative vs. discriminative. Discriminative models average **0.7694** macro-F1 against **0.6792** for generative models. The gap is consistent across all five representations. GaussianNB suffers most (0.57–0.58) because its independence and normality assumptions fit embeddings poorly.

A trap worth naming. `tfidf11_mnb` has the lowest neutral→negative rate (8.4%) but finds only 56% of genuinely negative posts. Its low rate reflects reluctance to predict “negative” at all — a model that never does so would score a perfect 0%. The two columns must always be read together.

9.4 Error Analysis

For `tfidf13_svm`, 36 of 250 neutral test posts (14.4%) were predicted negative:

Table 14: Error categories for neutral→negative misclassifications.

Tag	Count
Other	17
Mixed or contrastive tone	13
Contains SE jargon	5
Short question	1

Only 5 of 36 errors involve technical jargon — **on this corpus, the model’s errors are not primarily caused by technical vocabulary**, consistent with the vocabulary being nearly absent.

Implementation: `src/evaluation/evaluate.py`, `tools/run_final_evaluation.py`.

10 Explainability Analysis

10.1 LIME Explanations

LIME generates local explanations by perturbing a single input (randomly deleting words), querying the model on thousands of variants, and fitting a small linear model whose weights indicate each token’s contribution. Twenty carefully chosen posts are explained individually (5 correct negatives, 5 correct neutrals, 5 characteristic errors, 5 jargon-heavy), and aggregate weights are computed across 150 gold-neutral test posts.

Aggregate result: The tokens pushing hardest toward “negative” on neutral posts are:

afraid, me, extremely, i, m, am, !, was, very

These are **first-person and affective language**, not technical vocabulary. Zero SE jargon lexicon words occur three or more times in the neutral test posts — the fourth independent confirmation that the technical vocabulary is not in this corpus.

10.2 SHAP Explanations

SHAP provides theoretically grounded Shapley-value attributions. For the linear SVM model, `shap.LinearExplainer` provides exact values. The same 20 instances are explained for direct comparison with LIME.

Top SHAP negative-driving tokens:

hate, painful, line, me, pain, afraid, will not, can not, still

10.3 LIME–SHAP Agreement

The Spearman rank correlation between LIME and SHAP token rankings is $\rho = 0.872$ across 80 shared tokens. Two methods with fundamentally different mathematics — local linear surrogate vs. Shapley allocation — converge on the same account of model behaviour. This is strong evidence that both describe the model rather than their own sampling noise.

10.4 Stability Analysis

Across five random seeds on the 20 fixed posts, the mean Jaccard overlap of the top-5 tokens is **0.907** (sd 0.164). High but not 1.0 — individual explanations should be read as indicative, while aggregate tables provide the safer evidence.

Implementation: `src/evaluation/explain_lime.py`, `src/evaluation/explain_shap.py`.

11 Stress Test — The Central Result

Four independent analyses established that the Senti4SD corpus cannot answer the research question. A purpose-built 60-sentence stress set was constructed to do so:

- **40 harsh-but-neutral** sentences, e.g., “*Kill the process before restarting the server.*”
- **20 genuine complaints** (10 with SE jargon, 10 without), e.g., “*The documentation is worthless and the maintainers ignore every issue.*”
- **10 minimal pairs** using the same jargon word with opposite intent, e.g.,
Neutral: “*The app crashed after the update.*”
Negative: “*The app crashed again, this update is useless.*”

Sentences cover ≥ 20 SE lexicon words, maximum 5 sentences each, with lengths of 5–30 words.

Table 15: Stress-test results across models (sorted by accuracy).

Model	FA ↓	Neg R ↑	Jarg.	Plain	Pairs	Acc.
glove_gnb	2.5%	70%	90%	50%	90%	88.3%
tfidf13_svm	7.5%	75%	80%	70%	70%	86.7%
w2v_gnb	10.0%	80%	80%	80%	70%	86.7%
tfidf11_lr	7.5%	65%	70%	60%	60%	83.3%
tfidf12_lr	5.0%	50%	70%	30%	60%	80.0%
tfidf12_svm	5.0%	45%	70%	20%	60%	78.3%
tfidf13_mnb	30.0%	70%	90%	50%	50%	68.3%
w2v_lr	45.0%	65%	60%	70%	40%	58.3%
glove_svm	55.0%	75%	90%	60%	50%	55.0%
glove_lr	62.5%	85%	100%	70%	50%	53.3%
VADER	70.0%	75%	90%	60%	40%	45.0%

11.1 The Headline Result

VADER labels 70% of harsh-but-neutral SE sentences as negative. The best test model, `tfidf13_svm`, labels only **7.5%**, while still catching 75% of genuine complaints.

11.2 Outcome #3 — Lexical Bias Tracks Pretraining

Table 16: Mean false-alarm rate by representation — bias tracks general English exposure.

Representation	Mean False Alarm
VADER (general lexicon)	70%
GloVe (general English)	40%
Word2Vec (this corpus)	28%
TF-IDF (this corpus)	14%

The ordering tracks exactly how much general English each representation carries. Pretrained GloVe inherits the lexical bias from general English; representations learned on the StackOverflow corpus largely avoid it. This is the project’s direct answer to “*Do models rely on domain-appropriate cues or on general-language bias?*” — the bias is real, inherited from pretraining, and quantifiable.

Implementation: `data/stress_test/se_stress.csv`, `src/evaluation/stress_test.py`.

12 Web Application

A Flask-based web interface, **JargonSense**, is provided for interactive exploration of the retained pipeline. The server loads the `tfidf13_svm` model once at startup and exposes:

- `POST /api/predict` — accepts raw text, returns the predicted label, class probabilities, and LIME token weights (400 samples).
- `GET /api/examples` — 16 preloaded example sentences, each carrying a short label, a group, and its *expected* gold label.
- `GET /api/models` — the loaded model annotated with its test scores.
- `GET /api/results`, `GET /api/stress` — the master results table and the stress-test metrics. These are served for programmatic access; the page itself deliberately does not render them, so that the interface makes exactly one argument rather than restating the report.

The page shows the predicted label, the three class probabilities, and the sentence with each token shaded by its LIME weight. **The shading encodes *evidence for versus evidence against the prediction*, not negative versus positive.** Red and green already denote the two sentiment classes everywhere else in the interface, so reusing them for token weights would invert their meaning whenever the prediction is itself negative — a strongly negative-supporting token would be drawn in the colour of “positive”. Instead, tokens supporting the predicted class are drawn in the theme accent and tokens arguing against it in slate, with intensity proportional to $|w|$.

Because each example declares its gold label, the interface reports whether the model *agreed* with that label after every example click. This turns the demo into a claim that can fail in public rather than a showcase.

12.1 Held-Out Probe Sentences

Eight of the sixteen examples are **held-out probes** written for this report. They appear in neither Senti4SD nor the stress set, and every jargon term in them occurs **zero times** in the 3,031 training documents: *zombie*, *reap*, *poison*, *watchdog*, *orphan*, *panic*, *abort*, *deadlock*, *scheduler*. They therefore test the retained pipeline on vocabulary it has demonstrably never seen.

Table 17: Retained `tfidf13_svm` pipeline on the eight held-out probe sentences. Probabilities are negative / neutral / positive.

Probe	Gold	Predicted	Probabilities	Verdict
zombie	neutral	neutral	.04 / .95 / .00	correct
poison pill	neutral	neutral	.26 / .73 / .01	correct
watchdog	neutral	neutral	.14 / .82 / .04	correct
kernel panic	neutral	neutral	.12 / .88 / .00	correct
abort · calm	neutral	neutral	.05 / .83 / .12	correct
corruption	negative	negative	.54 / .45 / .02	correct
abort · angry	negative	<i>neutral</i>	.23 / .59 / .17	missed
praise	positive	<i>neutral</i>	.16 / .81 / .03	missed

The result is a compact replication of the project’s central finding. **All six harsh-but-neutral probes are classified neutral**, several with high confidence, despite being built entirely from words the model never encountered in training — a TF-IDF model cannot inherit a hostile prior for a word it has no feature for, which is exactly the mechanism Section 11 measures.

The two failures are equally informative, and they fail in the *opposite* direction to the one the literature warns about. Neither is a false alarm on jargon; both are **missed genuine sentiment**. “The service aborted my transaction again and lost two hours of work” carries its negativity in *again* and *lost two hours*, and “brilliant work on the lock ordering” carries its praise in *brilliant* (2 training occurrences). With the affective vocabulary too rare to have learned a strong weight, the surrounding technical prose pulls both sentences to neutral. This is the same weakness the test set records as 76.1% negative recall (Table 12), and it is the concrete cost of a model whose vocabulary is confined to 3,031 StackOverflow posts — precisely the deficit a pretrained contextual model would be expected to close (Section 8.3).

Implementation: `app/server.py`, launched via `python -m app.server` (the port is probed upward from 8000 because Windows reserves blocks of low ports for Hyper-V).

13 Limitations

1. **The corpus does not contain the phenomenon.** *fatal*, *hang*, and *abort* occur zero times; *kill* three times. Four independent analyses confirmed this. The central claim rests on the stress set, not on Senti4SD.
2. **The stress set was written by one person** who already knew what the models were expected to get wrong. The minimal pairs are a partial defence, but blind cross-annotation was not performed.
3. **Word2Vec quality is limited by corpus size** (3,031 documents). Its neighbour lists are largely noise, as anticipated in the proposal.
4. **LIME and SHAP are approximations.** Measured stability is 0.907 mean Jaccard, not 1.0. Individual explanations are indicative, not exact.
5. **Single corpus, single domain.** Whether the failure mode generalises to other technical domains (e.g., DevOps, bioinformatics) is untested.
6. **No contextual models.** A fine-tuned transformer (e.g., BERT) might substantially reduce or eliminate the lexical bias measured here. This was a deliberate trade of accuracy for exact explainability, argued in full in Section 8.3; it remains a limitation regardless of the justification.
7. **Annotation noise.** Fleiss’ $\kappa = 0.759$ indicates good but not perfect agreement. Some “errors” are disagreements the annotators also had.

14 Conclusion

A tuned TF-IDF (1,3) + calibrated linear SVM reaches **0.8275 test macro-F1** on the Senti4SD corpus, outperforming the VADER baseline by 0.12 points. Discriminative classifiers beat the generative ones decisively (mean 0.7694 vs. 0.6792), and TF-IDF beats both embedding families by a wide margin.

The project’s more significant contribution is methodological. The standard SE sentiment corpus does not contain the harsh technical vocabulary that motivates SE sentiment research, so the canonical claim cannot be tested on it — a fact that four independent analyses surfaced. When tested on purpose-built sentences, the bias is unambiguous: its magnitude tracks exactly how much general English the representation carries, from 70% false alarms for VADER down to 7.5% for corpus-trained TF-IDF+SVM.

LIME and SHAP confirm that the best model relies on first-person affective language rather than technical vocabulary, achieving a Spearman agreement of 0.872 between the two methods.

Reproducibility

Every number in this report is produced by a runnable script. The complete reproduction sequence:

```

pip install -r requirements.txt
python -m src.acquisition.download
python -m src.extraction.build_se           # corpus, Fleiss' kappa
python -m src.preprocessing.splits --domain se # frozen splits
python -m tools.run_baselines --domain se    # majority, VADER
python -m tools.run_ablation                # normalisation ablation
python -m src.features.features_tfidf       # TF-IDF vocabularies
python -m src.features.features_embed       # GloVe + Word2Vec
python -m src.modeling.models_tfidf         # 9 models
python -m src.modeling.models_embed         # 6 models
python -m tools.run_final_evaluation         # test-set results
python -m src.evaluation.explain_lime       # LIME
python -m src.evaluation.explain_shap       # SHAP
python -m src.evaluation.stress_test        # stress set
python -m pytest -q                         # 90 tests

```

The trained pipeline is saved to `models/se/tfidf13_svm.joblib`. The web interface is launched via `python -m app.server`.

References

- [1] Calefato, F., Lanubile, F., Maiorano, F., & Novielli, N. (2018). Sentiment Polarity Detection for Software Development. *Empirical Software Engineering*, 23(3), 1352–1382.
- [2] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD*, 1135–1144.
- [3] Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.
- [4] Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP*, 1532–1543.
- [5] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. *Advances in Neural Information Processing Systems*, 26.

- [6] Ahmed, T., Bosu, A., Iqbal, A., & Rahimi, S. (2017). SentiCR: A Customized Sentiment Analysis Tool for Code Review Interactions. *Proceedings of ASE*, 106–111.
- [7] Hutto, C. J. & Gilbert, E. (2014). VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. *Proceedings of ICWSM*, 8(1), 216–225.